

question 1

code:

```
#include <set>

class ECScheduler {
public:
    static ECScheduler& Instance() {
        static ECScheduler instance;
        return instance;
    }

    void StartTask(int id) {
        runningTasks.insert(id);
    }
    void StopTask(int id) {
        runningTasks.erase(id);
    }
    int GetNumRunTasks() const {
        return static_cast<int>(runningTasks.size());
    }

private:
    ECScheduler() = default;
    ~ECScheduler() = default;
    ECScheduler(const ECScheduler&) = delete;
    ECScheduler& operator=(const ECScheduler&) = delete;
    std::set<int> runningTasks;
};
```

output:

```
karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab9
$ clang++ ECSchedulerTest.cpp

karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab9
$ ./a.exe
Adding task 1: number of tasks: 1
Adding task 2: number of tasks: 2
Finally, number of tasks: 4
```

question 2

code:

```
// implement maze game
#include <vector>

// Room
class Room
{
public:
    virtual ~Room() = default;
    virtual int GetPrize() const { return 1; }
};

// Room with a bomb
class RoomWithBomb : public Room
{
public:
    int GetPrize() const override { return -10; }
};

// Enchanted room
class EnchantedRoom : public Room
{
public:
    int GetPrize() const override { return 20; } // Prize = 20
};

// maze game
class MazeGame
{
public:
    MazeGame() {}
    virtual ~MazeGame() {
        for (Room* room : rooms)
            delete room;
    }

    // create a maze with two rooms and that is it!
    void CreateMaze()
    {
        rooms.push_back(MakeRoom());
        rooms.push_back(MakeRoom());
    }

    // get the total prize amount for the rooms
    int GetTotPrize() const
    {
        int total = 0;
        for (const Room* room : rooms)
            total += room->GetPrize();
        return total;
    }

    virtual Room* MakeRoom() {
        return new Room();
    }

private:
    std::vector<Room*> rooms;
```

```

};

// bombed maze
class BombedMazeGame : public MazeGame
{
public:
    Room* MakeRoom() override {
        return new RoomWithBomb();
    }
};

// enchanted maze
class EnchantedMazeGame : public MazeGame
{
public:
    Room* MakeRoom() override {
        return new EnchantedRoom();
    }
};

```

output:

```

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab9
$ clang++ ECMazeGameTest.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab9
$ ./a.exe
game1 prize: 2
game2 prize: -20
game3 prize: 40

```